

Exercices architecture : circuits logiques et tables de vérité - Solution

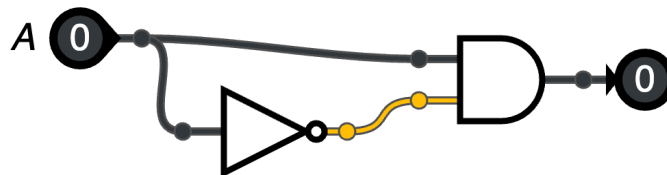
Exercice 1 : Circuit et table de vérité d'une fonction logique à 1 entrée

a) Dessinez le circuit et la table de vérité de la fonction : **A ET (NON A)**

Pour fabriquer le circuit :

- On ajoute autant d'entrées qu'il y a de variables dans la fonction. On les met à gauche.
- On applique les opérations en commençant par l'intérieur des parenthèses (comme en maths).
- On ajoute une sortie à droite, et on lui attribue éventuellement un nom.

Attention : l'ordre qu'on obtient n'est pas le même que dans la fonction.



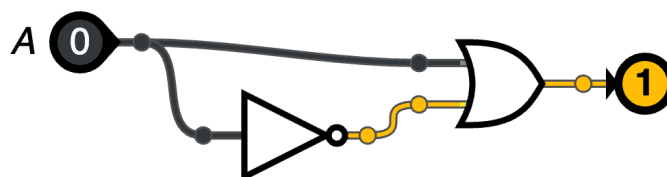
[→ ouvrir ce circuit dans le simulateur](#)

Pour faire la table de vérité :

- On la fabrique graduellement de gauche à droite.
- On utilise les colonnes déjà remplies pour calculer la colonne suivante.

A	NON A	A ET (NON A)
0	1	0
1	0	0

b) Dessinez le circuit et la table de vérité de la fonction : **A OU (NON A)**

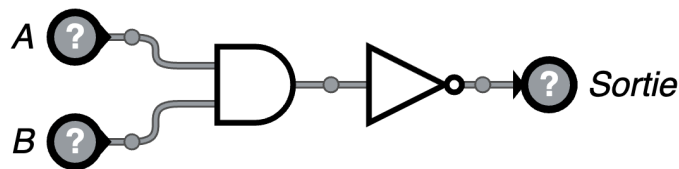


[→ ouvrir ce circuit dans le simulateur](#)

A	NON A	A OU (NON A)
0	1	1
1	0	1

Exercice 2 : Circuit et table de vérité d'une fonction logique à 2 entrées

Dessinez le circuit et la table de vérité de la fonction : **NON (A ET B)**

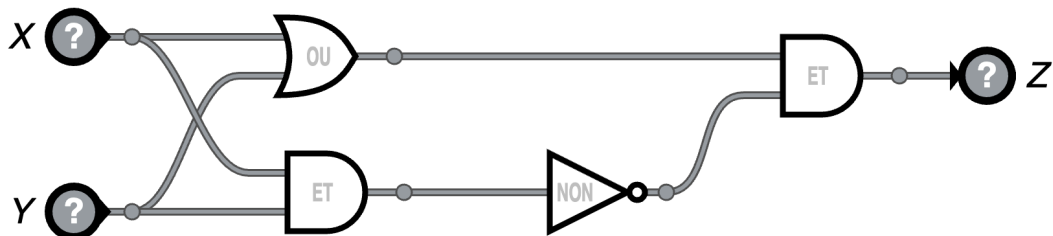


[→ ouvrir ce circuit dans le simulateur](#)

A	B	A ET B	NON (A ET B)
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

Exercice 3 : Table de vérité d'un circuit

a) Écrivez la table de vérité de ce circuit.



Les colonnes intermédiaires sont celles des fils qui sortent de chaque porte :

X	Y	X OU Y	X ET Y	NON (X ET Y)	Z
0	0	0	0	1	0
0	1	1	0	1	1
1	0	1	0	1	1
1	1	1	1	0	0

[→ ouvrir ce circuit dans le simulateur](#)

b) S'agit-il d'une fonction logique classique ? Si oui, laquelle ?

On reconnaît la porte OUX à 2 entrées : la sortie est à 1 quand une seule des entrées est à 1.

On peut aussi le voir directement sur le schéma. Z est à 1 quand :

- X OU Y sont à 1 (le haut du circuit), **ET**
- X et Y ne sont PAS à 1 en même temps (le bas du circuit).

Exercice 4 : Reconnaître une porte à partir de sa table de vérité

Chacune de ces trois tables de vérité est celle d'une porte de l'aide-mémoire. Pour chacune : donnez le nom de la porte, puis dessinez son circuit.

a)

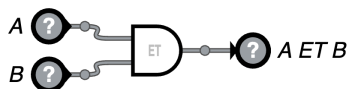
b)

c)

A	B	Sortie
0	0	0
0	1	0
1	0	0
1	1	1

Nom de la porte :

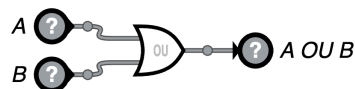
ET



A	B	Sortie
0	0	0
0	1	1
1	0	1
1	1	1

Nom de la porte :

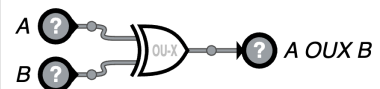
OU



A	B	Sortie
0	0	0
0	1	1
1	0	1
1	1	0

Nom de la porte :

OUX



Exercice 5 : Circuit d'une table de vérité proche d'une porte connue

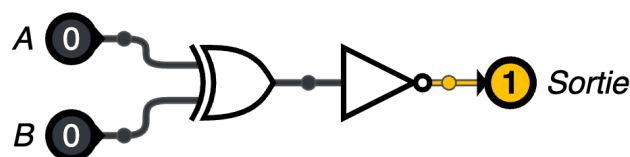
Cette table de vérité n'est celle d'aucune porte de l'aide-mémoire, mais elle en est très proche. Écrivez la fonction logique correspondante, puis dessinez son circuit.

A	B	Sortie
0	0	1
0	1	0
1	0	0
1	1	1

Fonction logique :

NON (A OUI-EX B)

La colonne Sortie est exactement l'inverse de celle du OUI-EX : elle vaut 1 partout où le OUI-EX vaut 0. Il suffit donc de brancher une porte NON à la sortie d'un OUI-EX. La sortie n'est à 1 que lorsque les deux entrées sont égales.



[→ ouvrir ce circuit dans le simulateur](#)

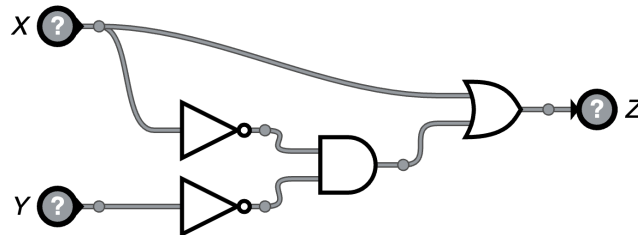
Exercice 6 : Circuit d'une table de vérité à 2 entrées

Dessinez le circuit correspondant à cette table de vérité. Il y a plusieurs circuits corrects : il suffit d'en trouver un.

X	Y	Z
0	0	1
0	1	0
1	0	1
1	1	1

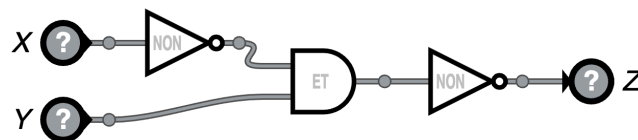
Le circuit que vous avez trouvé dépend de la manière dont vous avez réfléchi au problème.

Approche 1 : Z vaut 1 lorsque X ET Y sont les deux à 0, OU lorsque X est à 1.



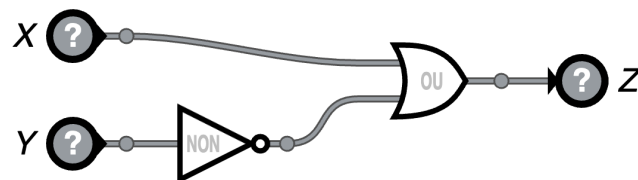
[→ ouvrir ce circuit dans le simulateur](#)

Approche 2 : Z vaut 0 seulement quand X = 0 ET Y = 1.



[→ ouvrir ce circuit dans le simulateur](#)

Approche 3 : Z vaut 1 lorsque X est à 1, OU lorsque Y est à 0.



[→ ouvrir ce circuit dans le simulateur](#)

Théorie : la forme normale disjonctive

Les trois approches ci-dessus demandent d'avoir une idée. En voici une quatrième qui n'en demande aucune : on peut toujours résoudre ces types de problèmes avec une approche très méthodique, la **forme normale disjonctive**. On fabrique le circuit de **droite à gauche** :

- On commence par connecter un OU à la sortie, avec autant d'entrées à ce OU qu'il y a de lignes dans la table.
- Pour chaque ligne du tableau où la sortie est à 1 :
 - on connecte une porte ET à gauche du OU final ;
 - on connecte chaque entrée à cette porte ET :
 - si l'entrée est à 1 dans le tableau, on la connecte directement ;
 - si l'entrée est à 0 dans le tableau, on insère une porte NON avant le ET.

On obtient un circuit correct, mais complexe. Par exemple pour l'exercice ci-dessus :

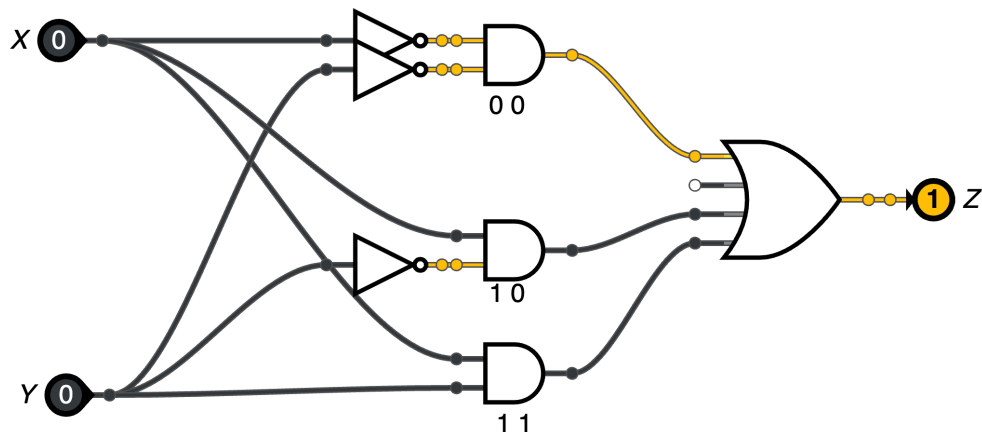
X	Y	Z
0	0	1
0	1	0
1	0	1
1	1	1

Paraphrase en français — Z est à 1 si :

$X = 0$ ET $Y = 0$

OU $X = 1$ ET $Y = 0$

OU $X = 1$ ET $Y = 1$



[→ ouvrir ce circuit dans le simulateur](#)

Exercice 7 : Circuit d'une table de vérité à 3 entrées

Dessinez le circuit correspondant à cette table de vérité.

A	B	C	Sortie
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

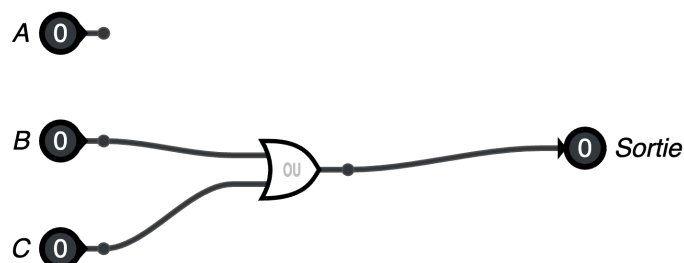
On peut réordonner les lignes sans que cela change la table de vérité. On regroupe les 0 et les 1 de la sortie pour faire apparaître dans les entrées de grands motifs qu'on pourrait reconnaître facilement. En analysant bien la table ainsi réordonnée, on voit que A n'a aucun impact sur la sortie :

A	B	C	Sortie
0	0	0	0
1	0	0	0
0	0	1	1
1	0	1	1
0	1	0	1
1	1	0	1
0	1	1	1
1	1	1	1

La table de vérité se réduit donc à :

B	C	Sortie
0	0	0
0	1	1
1	0	1
1	1	1

La sortie est à 1 quand B ou C sont à 1, d'où le circuit :

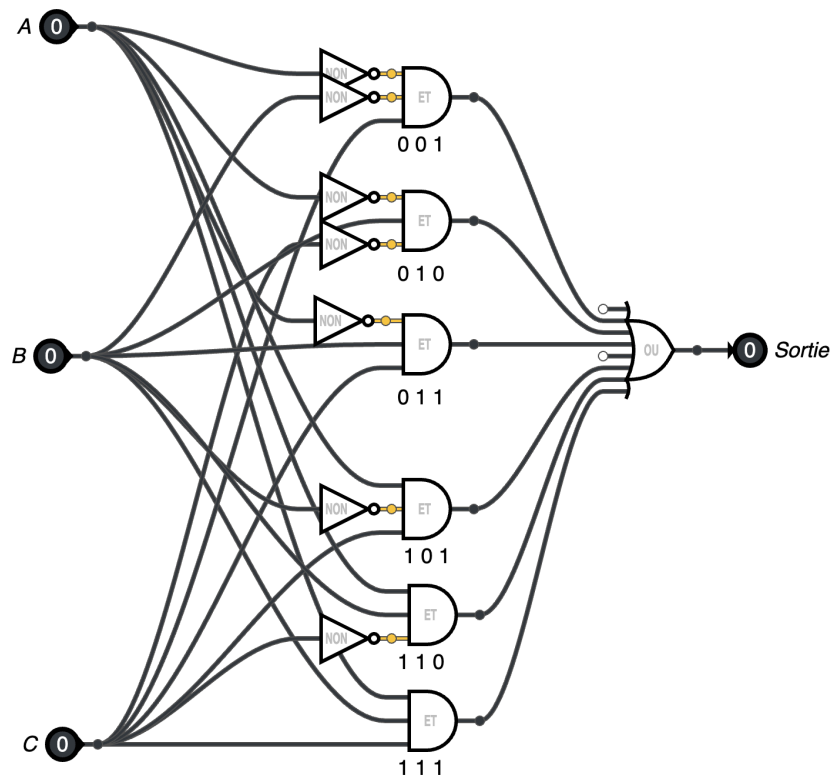


[→ ouvrir ce circuit dans le simulateur](#)

A n'est connecté d'aucune manière à la sortie dans ce circuit. C'est normal, puisque A n'a aucun impact sur la sortie dans la table de vérité.

La même chose avec la forme normale disjonctive :

Si on ne voit pas la simplification, on peut résoudre le problème avec la forme normale disjonctive. Mais on obtient un circuit très complexe :



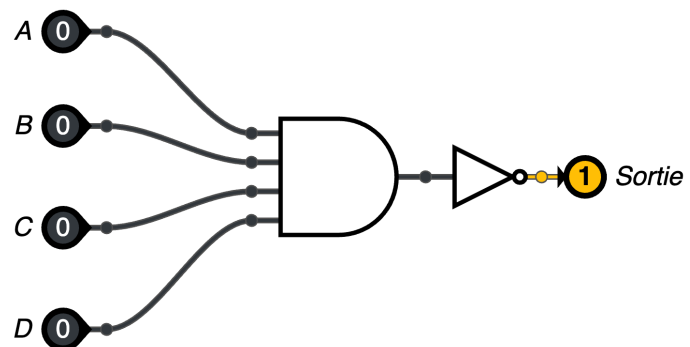
[→ ouvrir ce circuit dans le simulateur](#)

Exercice 8 : Circuit d'une table de vérité à 4 entrées

Dessinez le circuit correspondant à cette table de vérité.

A	B	C	D	Sortie
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Seule la dernière ligne est différente des autres : c'est ce cas-là qu'on va encoder.



[→ ouvrir ce circuit dans le simulateur](#)